PoRep(Proof-of-Replication) 개념과 구현

개요

PoRep(Proof-of-Replication)은 파일코인의 핵심 암호학적 증명 메커니즘 중 하나로, 저장 제공자가 고유한 데이터 복제본을 실제로 저장하고 있음을 증명하는 시스템입니다. 파일코인에서는 Stacked DRG(Depth Robust Graph) 기반의 PoRep을 사용하여 데이터의 물리적 저장과 시간-공간 트레이드오프 공격에 대한 저항성을 보장합니다.

Stacked DRG PoRep의 이론적 배경

1. PoRep의 목적과 필요성

PoRep은 다음과 같은 핵심 보안 속성을 제공합니다:

- **복제 증명**: 저장 제공자가 원본 데이터의 고유한 복제본을 생성했음을 증명
- **저장 증명**: 데이터가 실제로 물리적 저장 매체에 저장되어 있음을 증명
- **시간-공간 트레이드오프 저항성**: 계산을 통해 저장을 대체하려는 공격에 대한 방어

2. Stacked DRG의 구조적 특징

Stacked DRG는 여러 레이어로 구성된 그래프 구조를 사용합니다:

```
레이어 1: [노드1] [노드2] [노드3] ... [노드N]

↓ ↓ ↓ ↓

레이어 2: [노드1] [노드2] [노드3] ... [노드N]

↓ ↓ ↓ ↓
```

```
레이어 3: [노드1] [노드2] [노드3] ... [노드N]

...

레이어 L: [노드1] [노드2] [노드3] ... [노드N]
```

각 레이어의 노드는 다음 두 종류의 부모 노드를 가집니다:

- **기본 부모(Base Parents)**: 동일 레이어 내의 이전 노드들
- **확장 부모(Expansion Parents)**: 이전 레이어의 노드들

기본 DRG와 확장 그래프의 차이점

1. 기본 DRG (Depth Robust Graph)

기본 DRG는 단일 레이어 그래프로 다음과 같은 특성을 가집니다:

```rust
// storage-proofs-core/src/drgraph.rs에서 정의된 기본 DRG 구조
pub const BASE_DEGREE: usize = 6;


pub trait Graph<H: Hasher>: Debug + Clone + PartialEq + Eq {
    fn parents(&self, node: usize, parents: &mut [u32]) -> Result<()>;
    fn size(&self) -> usize;
    fn degree(&self) -> usize;
}
```

기본 DRG의 특징:

- 각 노드는 6개의 부모 노드를 가짐 (BASE_DEGREE = 6)
- 부모 노드는 버킷 샘플링 알고리즘으로 선택

- 단일 레이어 구조로 제한된 깊이

2. Stacked DRG (확장 그래프)

Stacked DRG는 기본 DRG를 다중 레이어로 확장한 구조입니다:

```
// storage-proofs-porep/src/stacked/vanilla/graph.rs에서 정의
pub const EXP_DEGREE: usize = 8;
pub(crate) const DEGREE: usize = BASE_DEGREE + EXP_DEGREE; // 6 + 8 = 14


pub struct StackedGraph<H, G> {
    expansion_degree: usize,
    base_graph: G,
    feistel_keys: [feistel::Index; 4],
    feistel_precomputed: FeistelPrecomputed,
    // ...
}
```

Stacked DRG의 장점:

- **증가된 깊이**: 여러 레이어를 통한 더 깊은 의존성 체인
- **향상된 보안**: 시간-공간 트레이드오프 공격에 대한 더 강한 저항성
- **확장 부모**: 이전 레이어와의 연결을 통한 추가적인 의존성

3. 부모 노드 생성 방식의 차이

기본 DRG 부모 생성:

```rust
// 기본 그래프에서는 버킷 샘플링 사용
impl<H: Hasher> Graph<H> for BucketGraph<H> {
    fn parents(&self, node: usize, parents: &mut [u32]) -> Result<()> {
        // 버킷 샘플링 알고리즘으로 부모 선택
        // 의사 랜덤하지만 결정적인 부모 선택
    }
}
```

Stacked DRG 부모 생성:

```rust
impl<H, G> Graph<H> for StackedGraph<H, G> {
    fn parents(&self, node: usize, parents: &mut [u32]) -> Result<()> {
        // 기본 부모 (동일 레이어)
        self.base_parents(node, &mut parents[..BASE_DEGREE])?;
        // 확장 부모 (이전 레이어)
        self.expanded_parents(
            node,
            &mut parents[BASE_DEGREE..BASE_DEGREE + EXP_DEGREE],
        )?;
        Ok(())
    }
}
```

Feistel 암호를 이용한 부모 노드 생성 메커니즘

1. Feistel 암호의 역할

Stacked DRG에서 확장 부모 노드 생성에는 Feistel 암호 기반의 의사 랜덤 순열이 사용됩니다:

```
// storage-proofs-core/src/crypto/feistel.rs
pub const FEISTEL_ROUNDS: usize = 3;
pub type Index = u64;
pub type FeistelPrecomputed = (Index, Index, Index);


pub fn permute(
num_elements: Index,
index: Index,
keys: &[Index],
precomputed: FeistelPrecomputed,
) -> Index {
        let mut u = encode(index, keys, precomputed);
        while u >= num_elements {
                u = encode(u, keys, precomputed)
        }
        u
}
```

2. Feistel 암호의 구조

Feistel 네트워크는 다음과 같은 3라운드 구조를 사용합니다:

```
입력 인덱스 → [L₀|R₀] 분할

라운드 1: L₁ = R₀, R₁ = L₀ ⊕ F(R₀, K₁)
라운드 2: L₂ = R₁, R₂ = L₁ ⊕ F(R₁, K₂)
라운드 3: L₃ = R₂, R₃ = L₂ ⊕ F(R₂, K₃)

출력 인덱스 ← [L₃|R₃] 결합
```

여기서 F는 라운드 함수로 Blake2b 해시를 사용합니다:

```rust
fn feistel(right: Index, key: Index, right_mask: Index) -> Index {
    let mut data: [u8; FEISTEL_BYTES] = [0; FEISTEL_BYTES];
    // right와 key를 결합하여 해시 입력 생성
    data[0] = (right >> 24) as u8;
    data[1] = (right >> 16) as u8;
    data[2] = (right >> 8) as u8;
    data[3] = right as u8;
    data[8] = (key >> 24) as u8;
    data[9] = (key >> 16) as u8;
    data[10] = (key >> 8) as u8;
    data[11] = key as u8;
    let hash_output = blake2b(&data);
```

```
        // 하위 비트를 마스크로 필터링하여 반환
        u64::from_le_bytes(hash_output.as_bytes()[..8].try_into().unwrap()) & ri
}
```

3. 확장 부모 생성 알고리즘

확장 부모는 다음과 같은 과정으로 생성됩니다:

```
impl<H, G> StackedGraph<H, G> {
        fn  correspondent(&self, node: usize, i: usize) -> u32 {
                // 노드와 부모 인덱스를 결합하여 고유한 시작점 생성
                let  a = (node * self.expansion_degree) as feistel::Index + i  a
                // Feistel 순열을 통한 의사 랜덤 변환
                let  transformed = feistel::permute(
                self.size() as feistel::Index * self.expansion_degree as feistel
                a,
                &self.feistel_keys,
                self.feistel_precomputed,
                );
                // 변환된 값을 노드 인덱스로 매핑
                transformed  as u32 / self.expansion_degree as u32
        }
        pub  fn  generate_expanded_parents(&self, node: usize, expanded_parents:
                for (i, el) in  expanded_parents.iter_mut().enumerate() {
```

```
                        *el = self.correspondent(node, i);

                }

        }

}
```

4. Feistel 키 생성

Feistel 암호에 사용되는 키는 PoRep ID로부터 결정적으로 생성됩니다:

```
pub fn derive_feistel_keys(porep_id: PoRepID) -> [u64; 4] {

        let mut feistel_keys = [0u64; 4];

        let raw_seed = derive_porep_domain_seed(FEISTEL_DST, porep_id);

        feistel_keys[0] = u64::from_le_bytes(raw_seed[0..8].try_into().unwrap())

        feistel_keys[1] = u64::from_le_bytes(raw_seed[8..16].try_into().unwrap()

        feistel_keys[2] = u64::from_le_bytes(raw_seed[16..24].try_into().unwrap(

        feistel_keys[3] = u64::from_le_bytes(raw_seed[24..32].try_into().unwrap(

        feistel_keys

}
```

레이블 생성과 인코딩 과정

1. 레이어별 레이블 생성

각 레이어의 노드는 부모 노드들의 데이터를 해시하여 레이블을 생성합니다:

```rust
// 첫 번째 레이어 (기본 부모만 사용)
pub fn create_label<H: Hasher>(
graph: &StackedBucketGraph<H>,
cache: Option<&mut ParentCache>,
replica_id: &[u8],
layer_labels: &mut [u8],
layer: usize,
node: usize,
) -> Result<()> {
        let mut parents = [0u32; BASE_DEGREE];
        graph.base_parents(node, &mut parents)?;
        // 부모 데이터를 해시하여 레이블 생성
        let label = hash_parents_and_replica_id(parents, replica_id, node);
        write_node_label(layer_labels, node, &label);
        Ok(())
}

// 두 번째 레이어 이후 (기본 + 확장 부모 사용)
pub fn create_label_exp<H: Hasher>(
graph: &StackedBucketGraph<H>,
cache: Option<&mut ParentCache>,
replica_id: &[u8],
```

```
exp_parents_data: &[u8],

layer_labels: &mut [u8],

layer: usize,

node: usize,

) -> Result<()> {

        let mut parents = [0u32; DEGREE]; // BASE_DEGREE + EXP_DEGREE

        graph.parents(node, &mut parents)?;

        // 기본 부모와 확장 부모 데이터를 모두 사용하여 레이블 생성

        let label = hash_all_parents_and_replica_id(parents, replica_id, exp_pa

        write_node_label(layer_labels, node, &label);

        Ok(())

}
```

2. 해시 기반 레이블 계산

레이블 생성에는 SHA256 해시가 사용되며, 여러 라운드의 해시 연산을 통해 보안성을 강화합니다:

```
fn copy_parents_data_inner_exp(

&self,

cache_parents: &[u32],

base_data: &[u8],

exp_data: &[u8],

mut hasher: Sha256,

) -> [u8; 32] {
```

```
        // 부모 노드 데이터 읽기 (캐시 최적화)
        let parents = [
            read_node(0, cache_parents, base_data), // 기본 부모 1-6
            read_node(1, cache_parents, base_data),
            // ... (BASE_DEGREE개)
            read_node(6, cache_parents, exp_data), // 확장 부모 1-8
            read_node(7, cache_parents, exp_data),
            // ... (EXP_DEGREE개)
        ];
        // 다중 라운드 해시 연산
        hasher.input(&parents); // 라운드 1
        hasher.input(&parents); // 라운드 2
        hasher.input(&parents[..8]); // 라운드 3 (일부)
        hasher.finish_with(parents[8]) // 최종 라운드
}
```

성능 최적화 기법

1. 부모 캐시 시스템

빈번한 부모 노드 접근을 최적화하기 위해 캐시 시스템을 사용합니다:

```
pub struct ParentCache {
        cache_size: u32,
```

```
        cache_entries: u32,
        cache: Vec<[u32; DEGREE]>,
}

impl ParentCache {
        pub fn read(&mut self, node: u32) -> Result<&[u32; DEGREE]> {
                let cache_index = node % self.cache_size;
                if self.cache[cache_index as usize][0] != node {
                        // 캐시 미스 - 부모 노드 계산 후 캐시에 저장
                        let mut parents = [0u32; DEGREE];
                        self.graph.parents(node as usize, &mut parents)?;
                        self.cache[cache_index as usize] = parents;
                }
                Ok(&self.cache[cache_index as usize])
        }
}
```

2. 메모리 프리페치

부모 노드 데이터 접근 시 CPU 캐시 효율성을 위한 프리페치:

```
#[inline]
fn prefetch(parents: &[u32], data: &[u8]) {
        for parent in parents {
```

```
                let  start = *parent  as usize * NODE_SIZE;

                let  end = start + NODE_SIZE;

                // CPU에게 해당 메모리 영역을 미리 캐시로 로드하도록 힌트

                prefetch!(data[start..end].as_ptr() as *const i8);

        }

}
```

보안 속성과 공격 저항성

1. 시간-공간 트레이드오프 저항성

Stacked DRG의 다층 구조는 다음과 같은 공격을 방어합니다:

- **저장 회피 공격**: 데이터를 저장하지 않고 계산으로 대체하려는 시도
- **병렬 처리 공격**: 여러 증명을 동시에 생성하려는 시도
- **메모리 절약 공격**: 일부 데이터만 저장하고 나머지를 재계산하려는 시도

2. 암호학적 가정

PoRep의 보안성은 다음 가정들에 기반합니다:

- **해시 함수의 랜덤 오라클 모델**: SHA256과 Blake2b의 암호학적 안전성
- **Feistel 네트워크의 의사 랜덤성**: 3라운드 Feistel이 충분한 의사 랜덤 순열 제공
- **그래프의 확장 속성**: DRG의 깊이 견고성과 확장 속성

3. 검증 가능성

생성된 PoRep은 다음과 같이 효율적으로 검증 가능합니다:

- **결정적 재생성**: 동일한 입력으로부터 동일한 결과 보장
- **부분 검증**: 전체 데이터를 다시 처리하지 않고도 증명 검증 가능
- **공개 검증**: 비밀 정보 없이도 누구나 증명 검증 가능

결론

Stacked DRG PoRep은 파일코인의 저장 증명 시스템의 핵심으로, 기본 DRG의 한계를 극복하고 강화된 보안 속성을 제공합니다. Feistel 암호를 통한 의사 랜덤 부모 생성, 다층 레이어 구조, 그리고 다양한 성능 최적화 기법을 통해 실용적이면서도 안전한 저장 증명 메커니즘을 구현합니다.

이러한 설계는 저장 제공자가 실제로 데이터를 물리적으로 저장하도록 강제하면서도, 증명 생성과 검증 과정에서 합리적인 성능을 보장합니다. 또한 다양한 공격 벡터에 대한 저항성을 제공하여 파일코인 네트워크의 보안성과 신뢰성을 뒷받침합니다.